

Topic 5 神经网络（Neural Network）

Part 1：神经网络简介

魏准
浙江大学，信息与电子工程学院
2026

总体内容

主要知识点

- 神经网络的节点（Nodes）
- 神经网络的层(Layers)
- 神经网络监督学习
- 训练一个单层神经网络
- 普适化Delta 规则
- 权重更新计算规则：SGD, Batch, and Mini Batch
- 实例Delta 规则训练
  - ◆ 实例5-1 SGD
  - ◆ 实例5-1 Batch
  - ◆ 实例5-1 SGD vs Batch

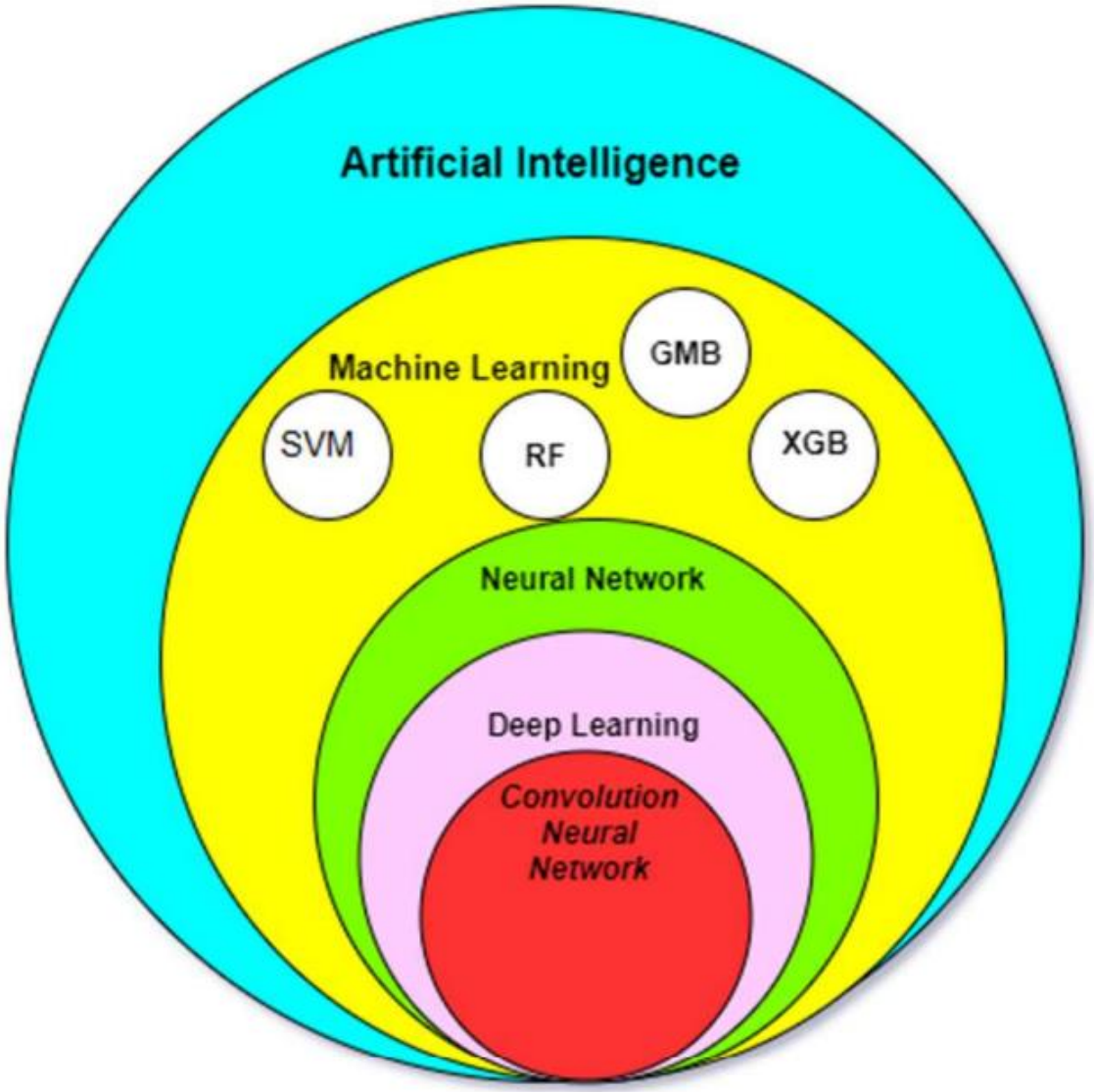
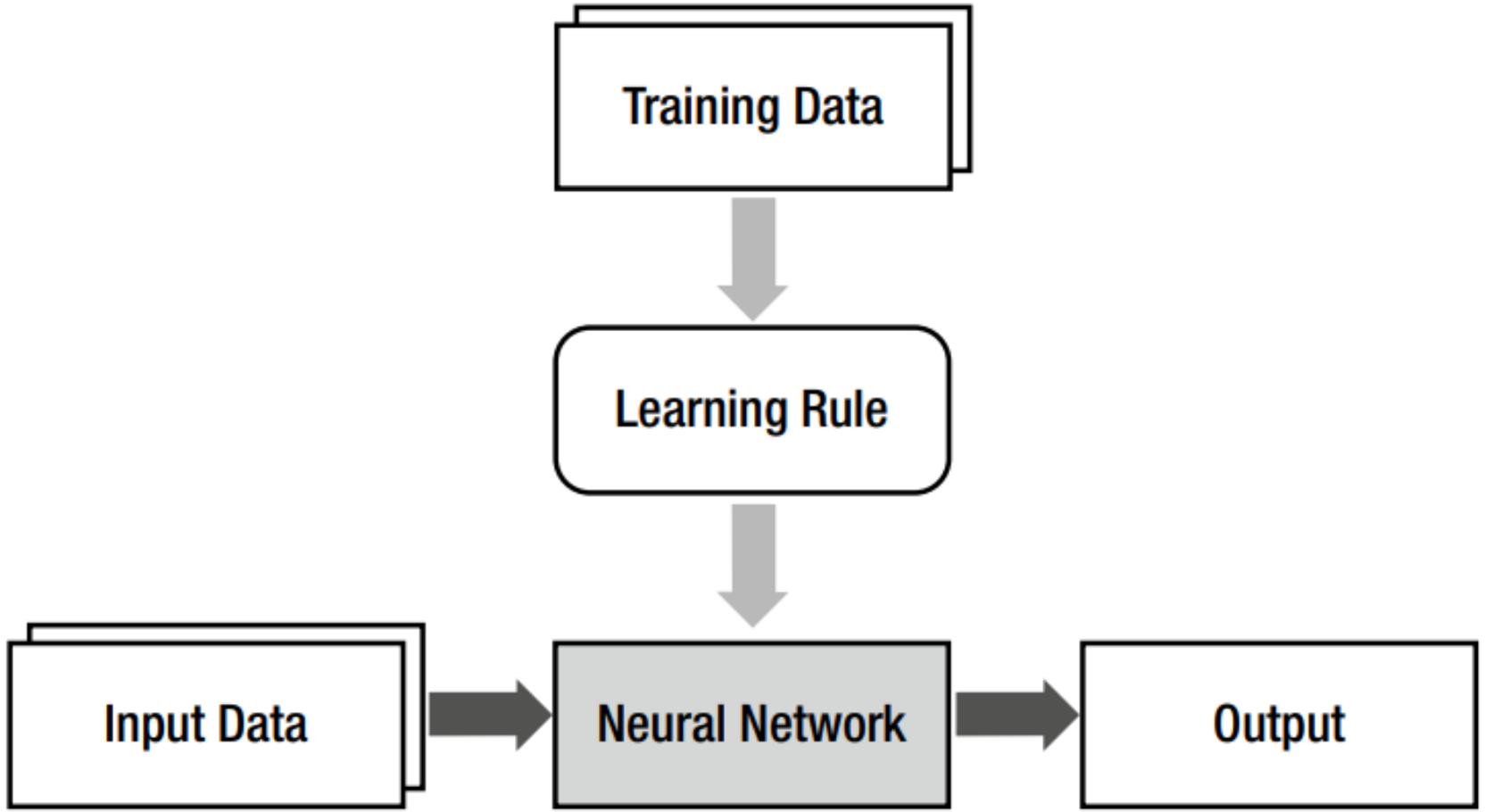


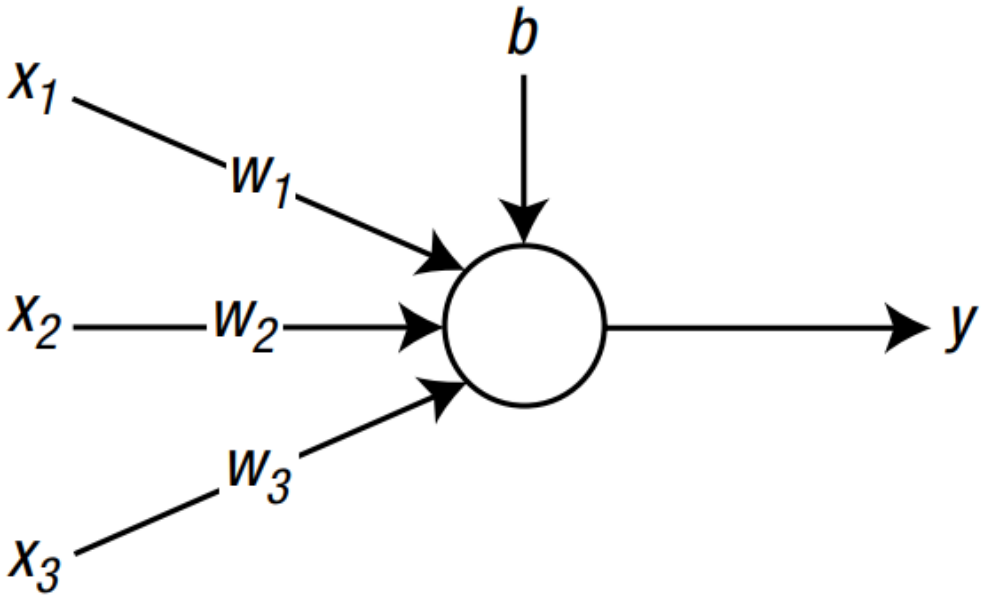
FIGURE 1 | The relationship between the most common AI methods in medicine. SMV, Support Vector Machine; RF, Random forest algorithm; GBM, Gradient Boosting Machines; XGB, XGBoost.

Sotoudeh, Houman, et al. "Artificial intelligence in the management of glioma: era of personalized medicine." Frontiers in oncology 9 (2019): 768.



Phil Kim, 2017. MATLAB Deep Learning: With Machine Learning, Neural Networks and Artificial Intelligence.

1.1：神经网络的节点（Nodes）



一个接受了3个输入的节点

$$v = (w_1 \times x_1) + (w_2 \times x_2) + (w_3 \times x_3) + b$$

矩阵形式

$$v = wx + b$$

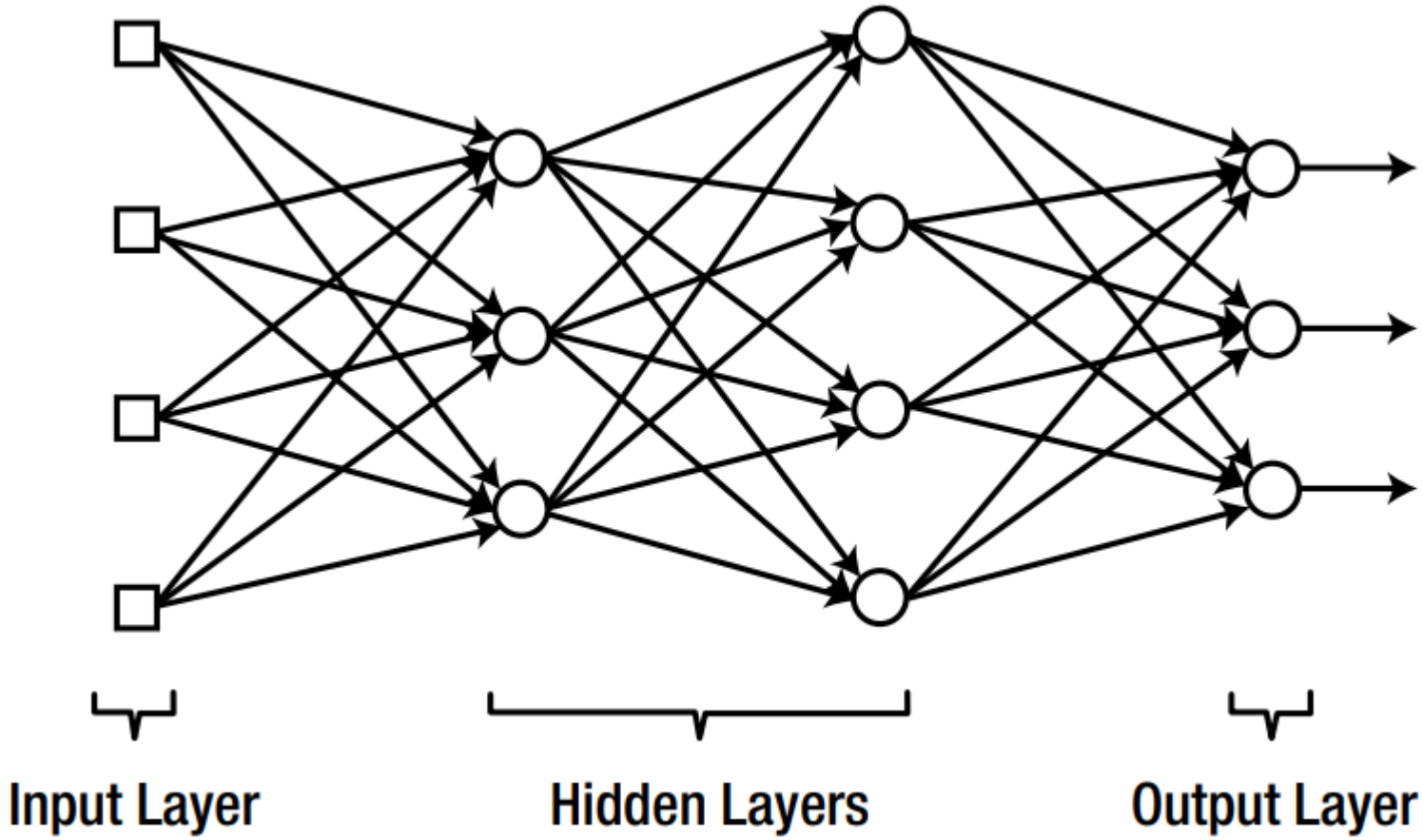
$$w = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

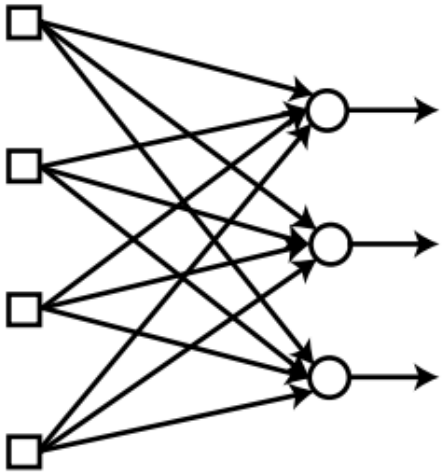
添加激活函数

$$y = \varphi(v)$$

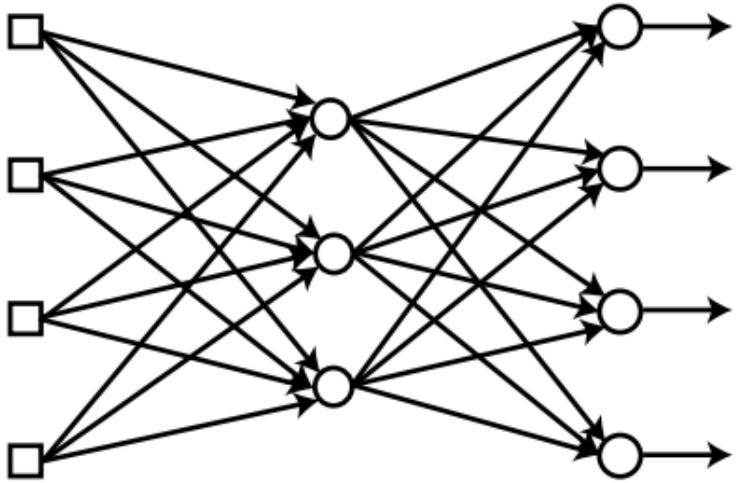
$$y = \varphi(v) = \varphi(wx + b)$$

1.2：神经网络的层(Layers)

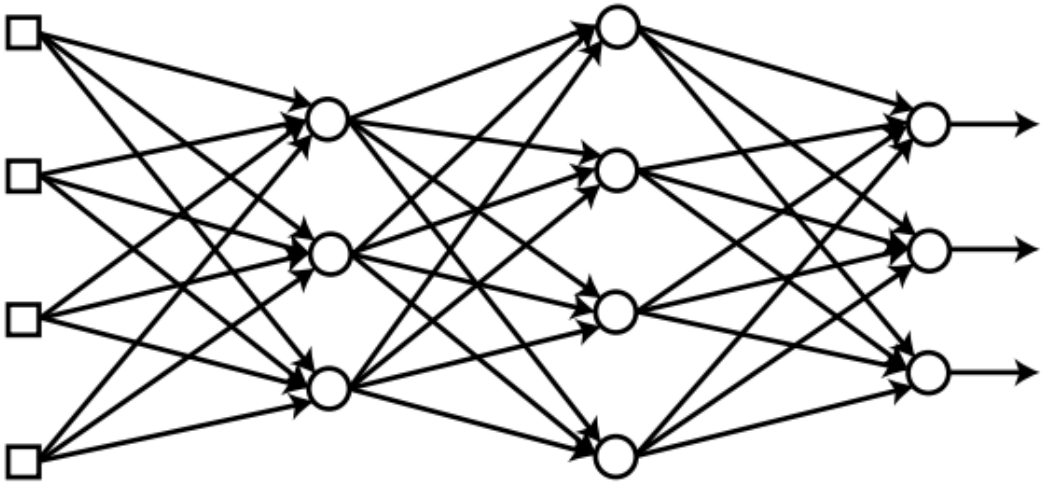




Single-layer Neural Network

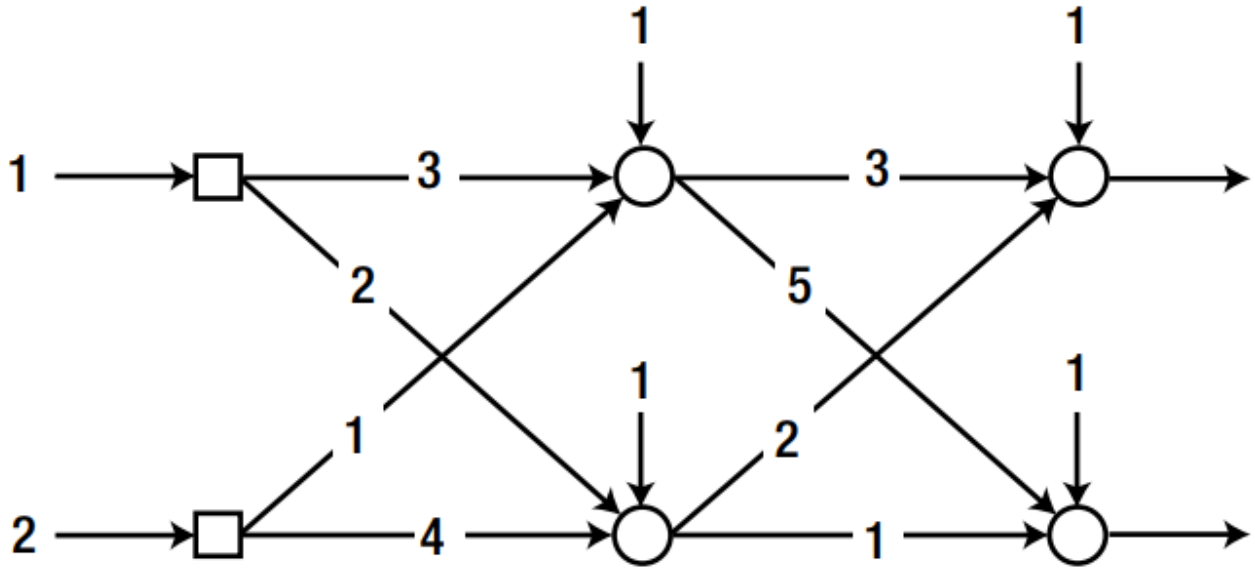


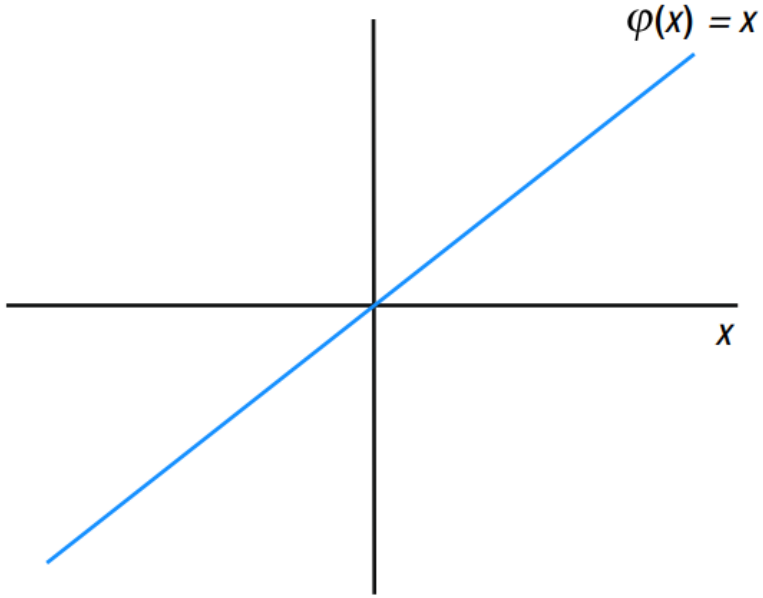
(Shallow) Multi-layer Neural Network



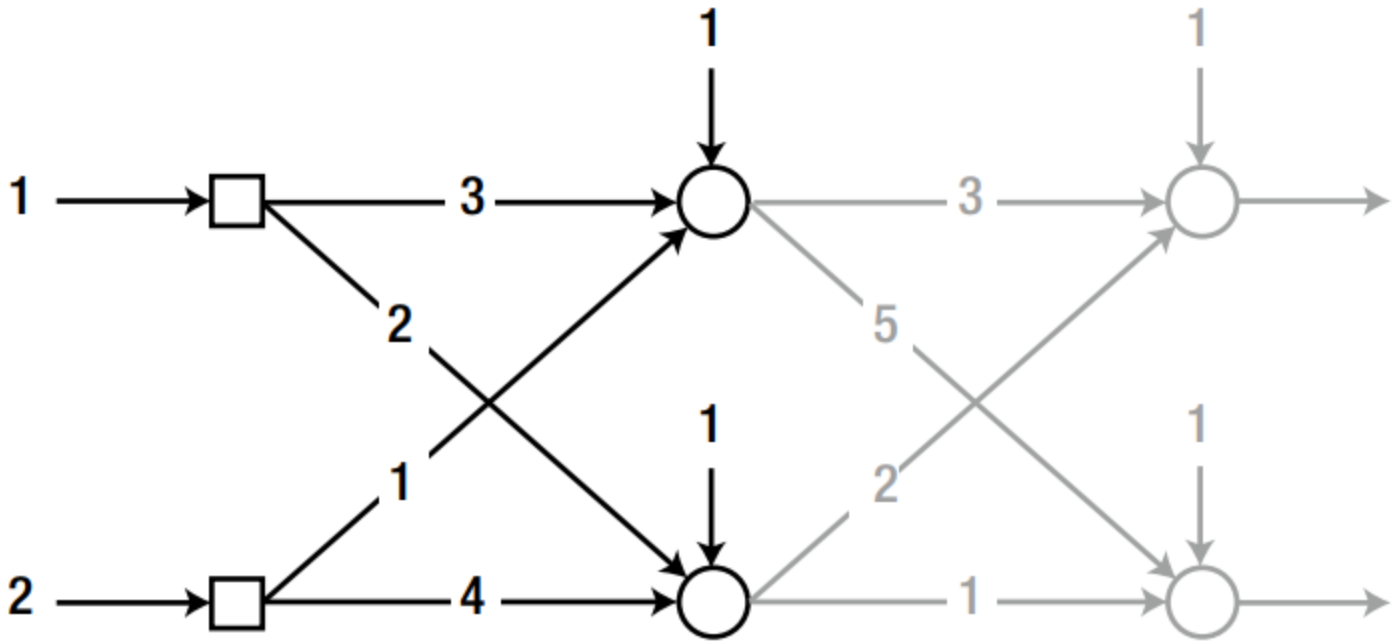
Deep Neural Network

例子：单层隐藏层网络





计算隐藏层输出



计算第一个节点输出

Weighted sum: $v = (3\times1) + (1\times2) + 1 = 6$

Output: $y = \varphi(v) = v = 6$

计算第二个节点输出

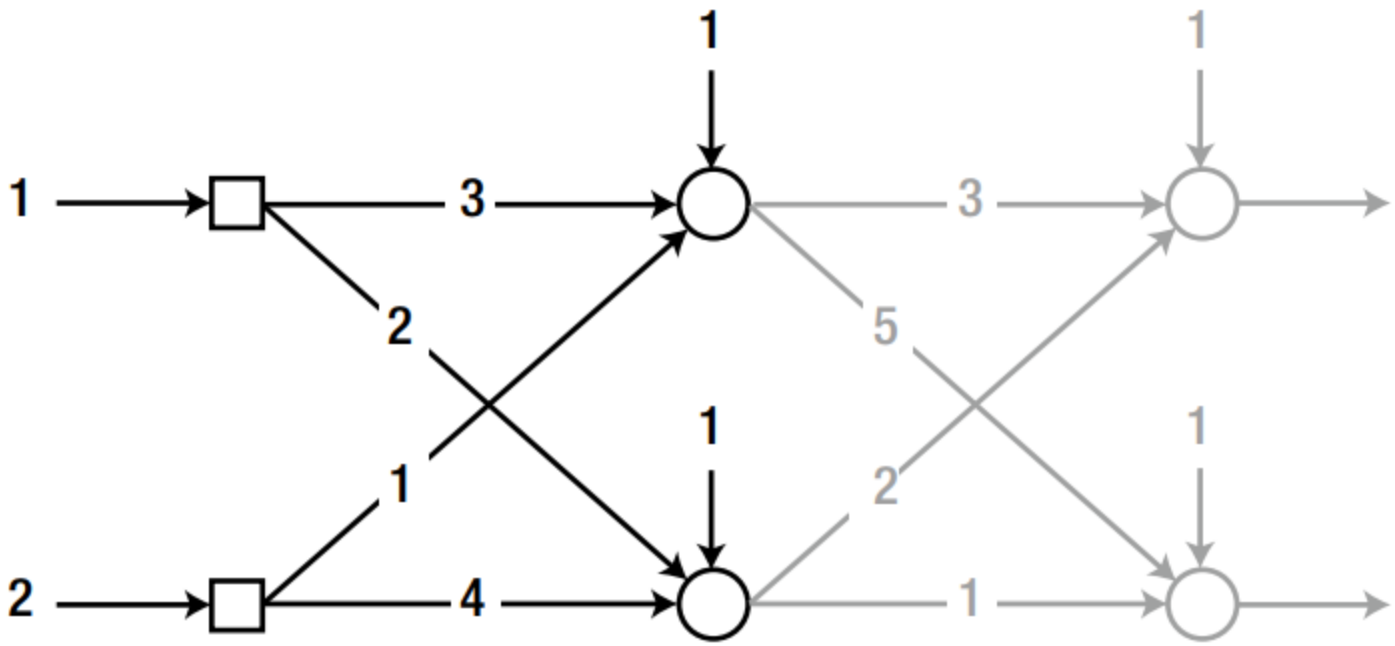
Weighted sum: $v = (2\times1) + (4\times2) + 1 = 11$

Output: $y = \varphi(v) = v = 11$

矩阵形式:

$$v = \begin{bmatrix} 3\times1+1\times2+1 \\ 2\times1+4\times2+1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 11 \end{bmatrix}$$

计算隐藏层输出



计算第一个节点输出

Weighted sum: $v = (3\times1) + (1\times2) + 1 = 6$

Output: $y = \varphi(v) = v = 6$

计算第二个节点输出

Weighted sum: $v = (2\times1) + (4\times2) + 1 = 11$

Output: $y = \varphi(v) = v = 11$

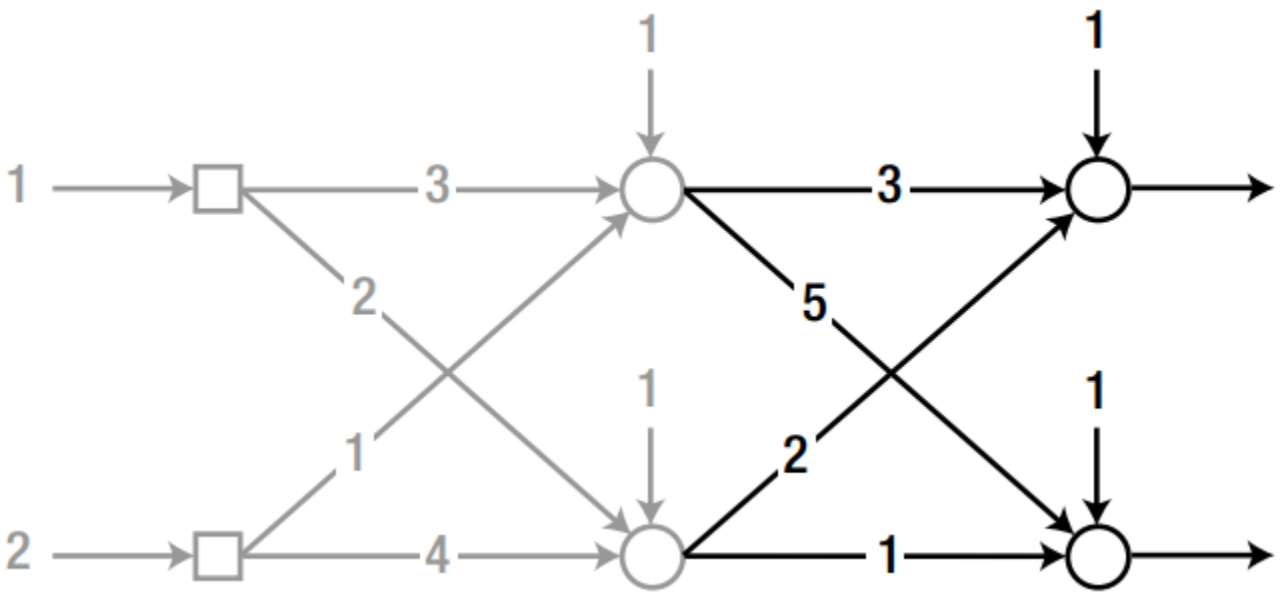
矩阵形式:

$$v = \begin{bmatrix} 3\times1+1\times2+1 \\ 2\times1+4\times2+1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 11 \end{bmatrix}$$

权重: w

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

计算输出层输出



矩阵形式:

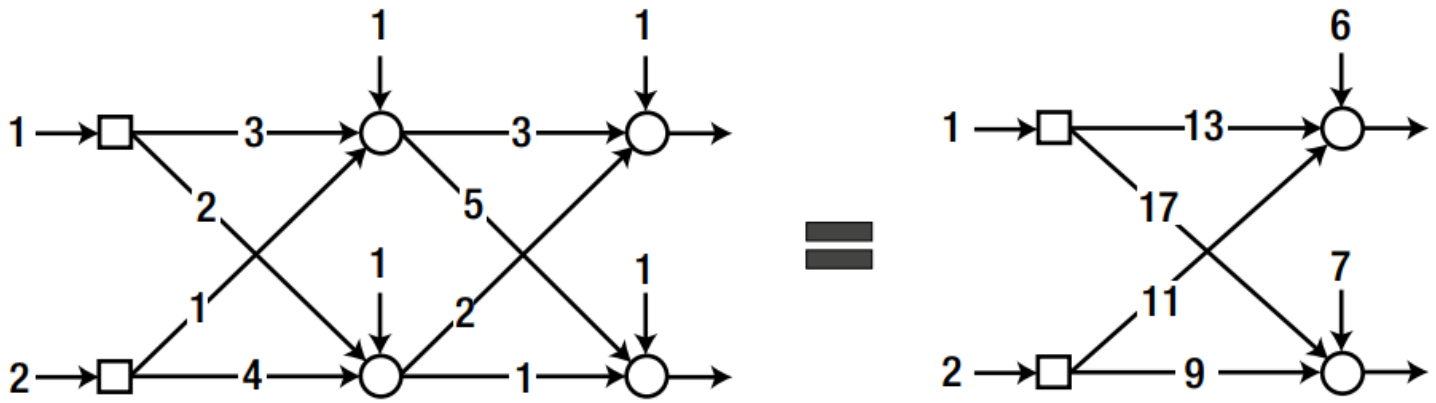
加权和 $$v = \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

输出 $$y = \varphi(v) = v = \begin{bmatrix} 41 \\ 42 \end{bmatrix}$$

整个过程

$$
\begin{aligned}
v &= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 11 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \left(\begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 & 2 \\ 5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\
&= \begin{bmatrix} 13 & 11 \\ 17 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 6 \\ 7 \end{bmatrix}
\end{aligned}
$$

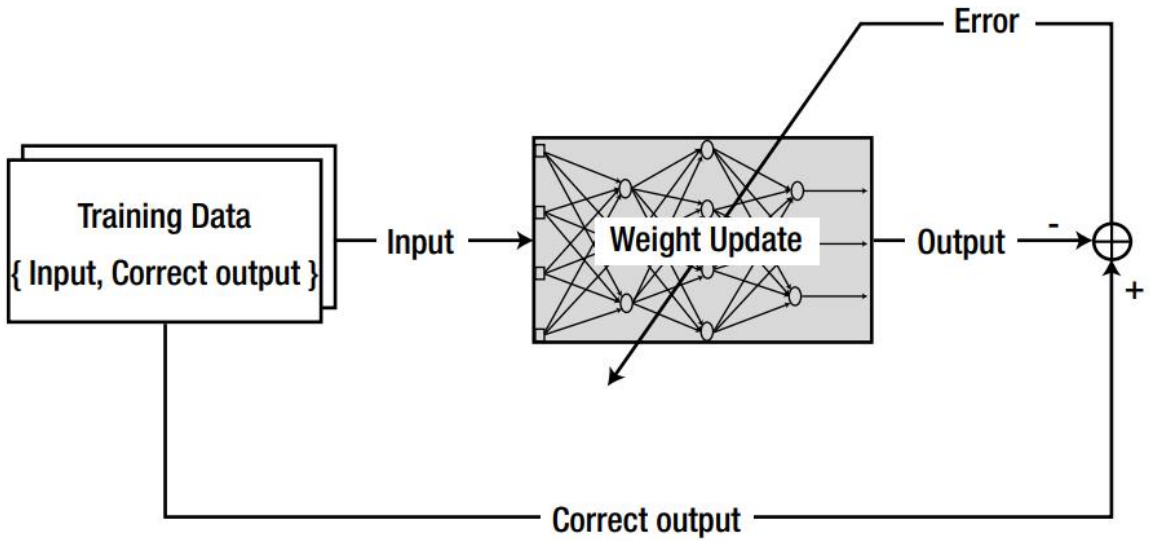
输入



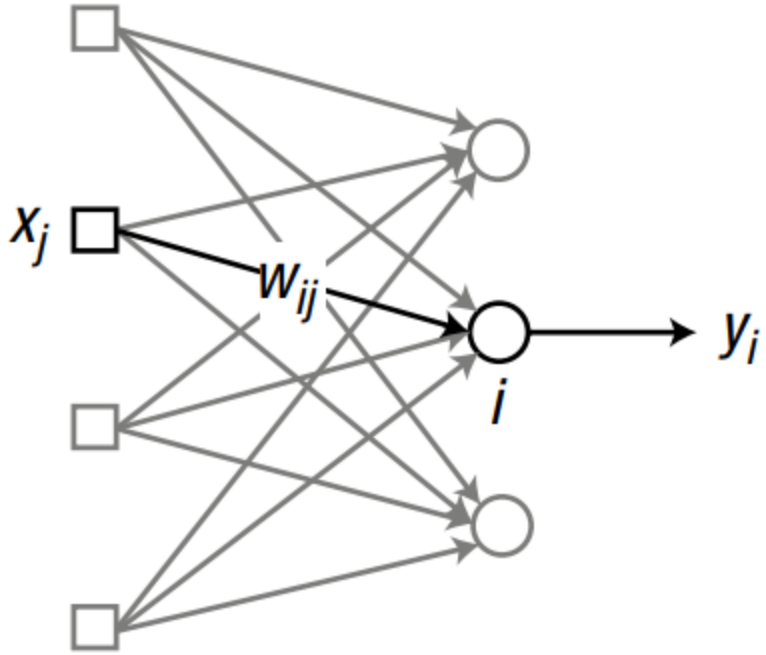
隐藏层不起作用！！？

1.3：神经网络监督学习

1. 初始化权重；
2. 通过训练数据对（输入、准确解），计算NN的输出、误差；
3. 调整权重；
4. 重复2-3，直到遍历所有训练数据；



1.4：训练一个单层神经网络



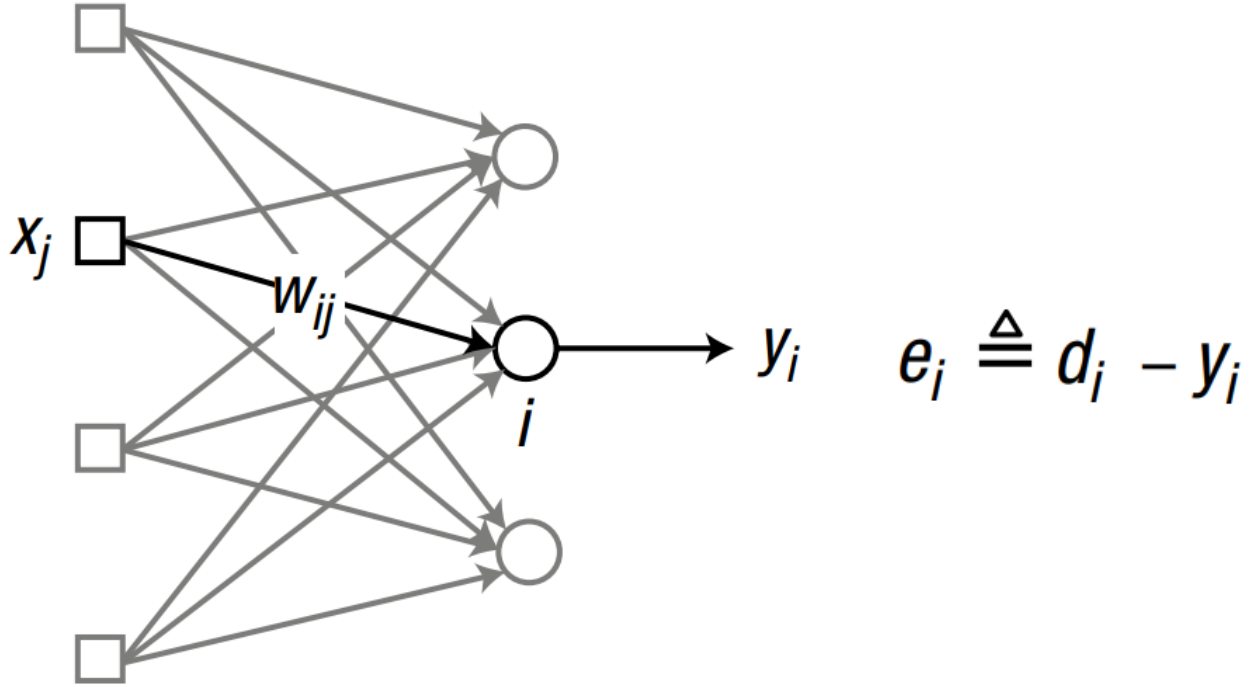
$$e_i \triangleq d_i - y_i$$

准确解

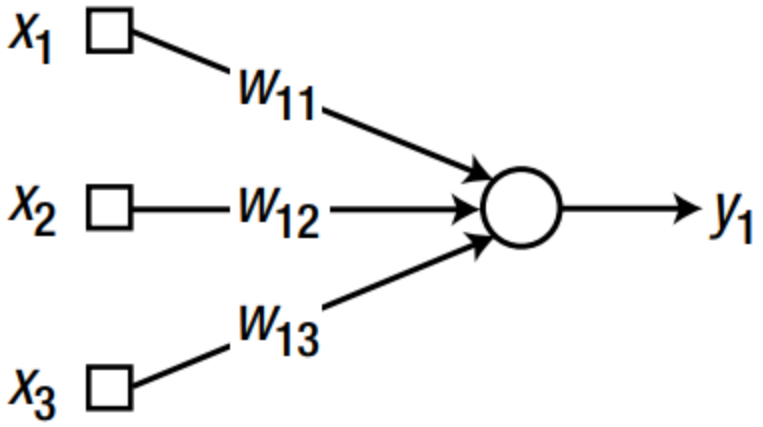
Delta 规则(Delta rule/Adaline /Widrow-Hoff rule)

$$w_{ij} \leftarrow w_{ij} + \alpha e_i x_j$$

- x_j表示上一级节点j的输出;
- e_i节点i的输出误差;
- W_{ij}节点j和节点i之间的权重;
- $0 < \alpha \leq 1$表示学习率



例子:



$$e_1 = d_1 - y_1$$

$$w_{11} \leftarrow w_{11} + \alpha e_1 x_1$$
$$w_{12} \leftarrow w_{12} + \alpha e_1 x_2$$
$$w_{13} \leftarrow w_{13} + \alpha e_1 x_3$$

Delta 规则步骤：

1. 初始化权重；
2. 通过训练数据对（输入、准确解），计算NN的输出、误差；

$$e_i = d_i - y_i$$

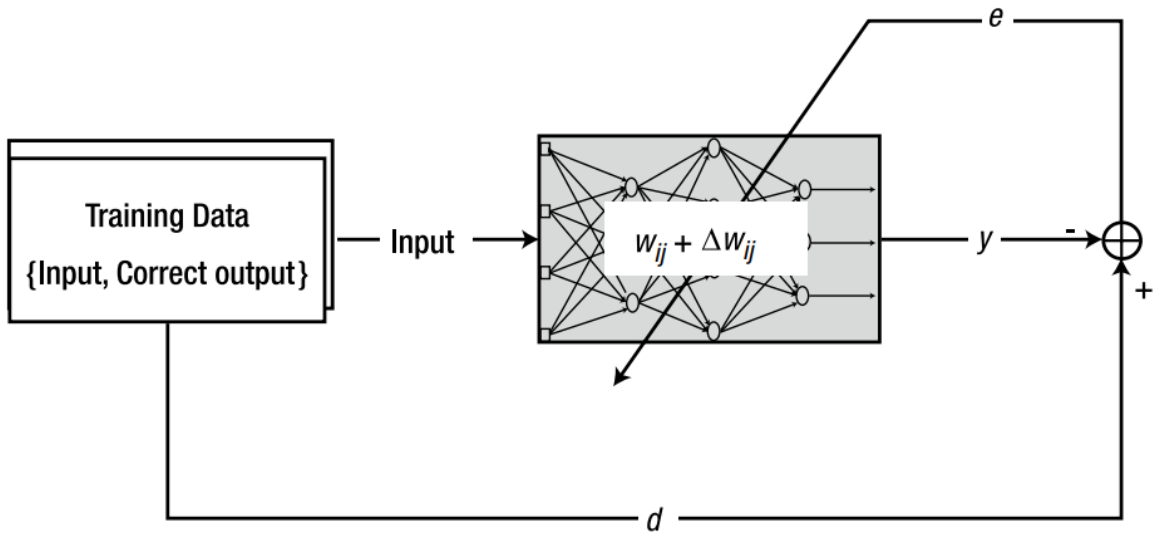
3. 根据下列公式，计算误差更新值

$$\Delta w_{ij} = \alpha e_i x_j$$

4. 根据下列公式调整权重

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

5. 重复2-4，直到遍历所有训练数据
6. 重复2-5直到网络达到训练要求；



1.5：普适化Delta 规则

$$w_{ij} \leftarrow w_{ij} + \alpha e_i x_j \longrightarrow w_{ij} \leftarrow w_{ij} + \alpha \delta_i x_j$$

其中

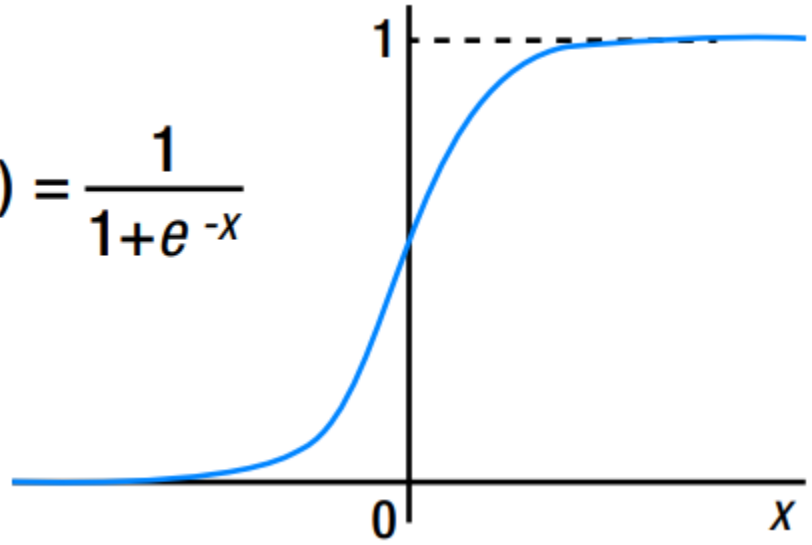
$$\delta_i = \varphi'(v_i) e_i$$

- e_i节点i的输出误差;
- v_i节点i加权输出;
- φ‘表示节点i的激活函数的导数;

如果 $\varphi(x) = x$ 则 $\delta_i = e_i$

Sigmoid激活函数：

$$\varphi(x) = \frac{1}{1+e^{-x}}$$



对激活函数求导，很容易得到：

$$\varphi'(x) = \varphi(x)\left(1-\varphi(x)\right)$$

从而

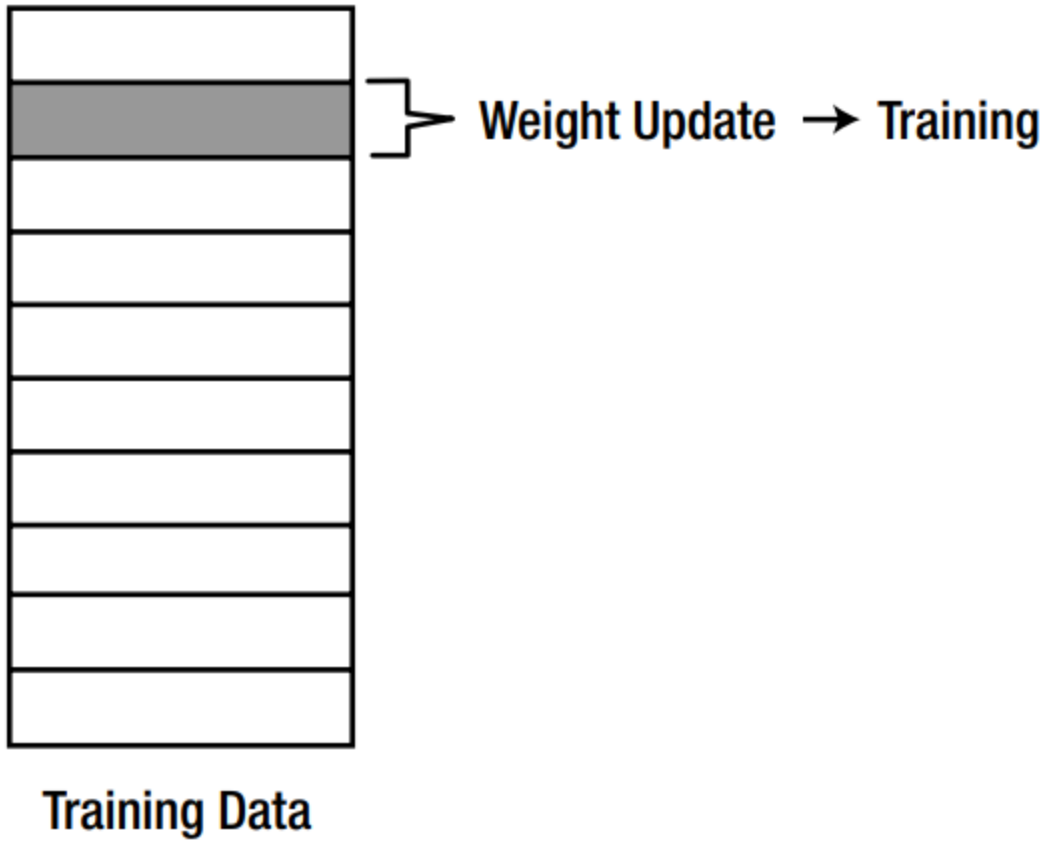
$$\delta_i = \varphi'(v_i)e_i = \varphi(v_i)\left(1-\varphi(v_i)\right)e_i$$

所以对于Sigmoid激活函数，普适化Delta 规则更新规则为：

$$w_{ij} \leftarrow w_{ij} + \alpha\,\varphi(v_i)\left(1-\varphi(v_i)\right)e_i x_j$$

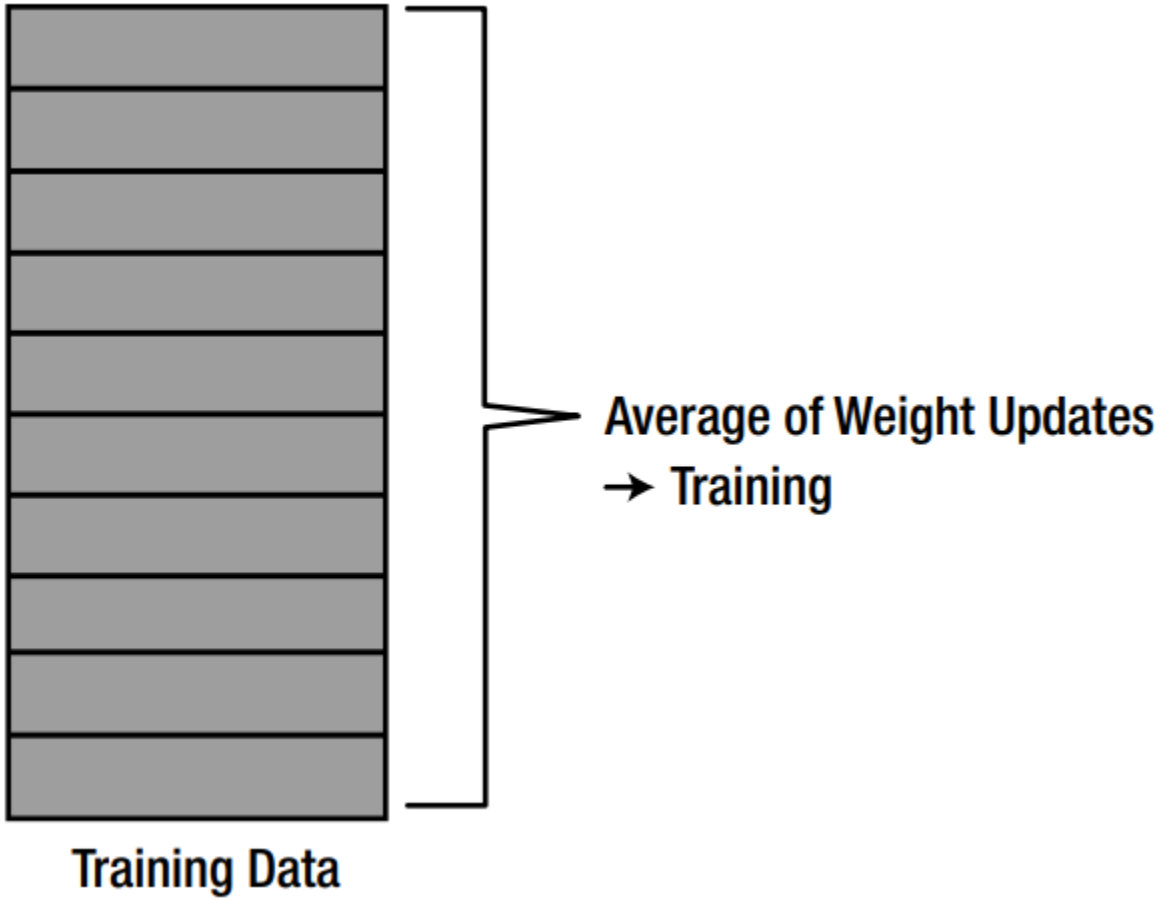
1.6：权重更新计算规则：SGD, Batch, and Mini Batch

随机梯度下降，Stochastic Gradient Descent (SGD)：计算每一个训练数据的error（每次随机选取），并立即更新权重。比如如果有100个训练数据，SGD则调整100次权重。



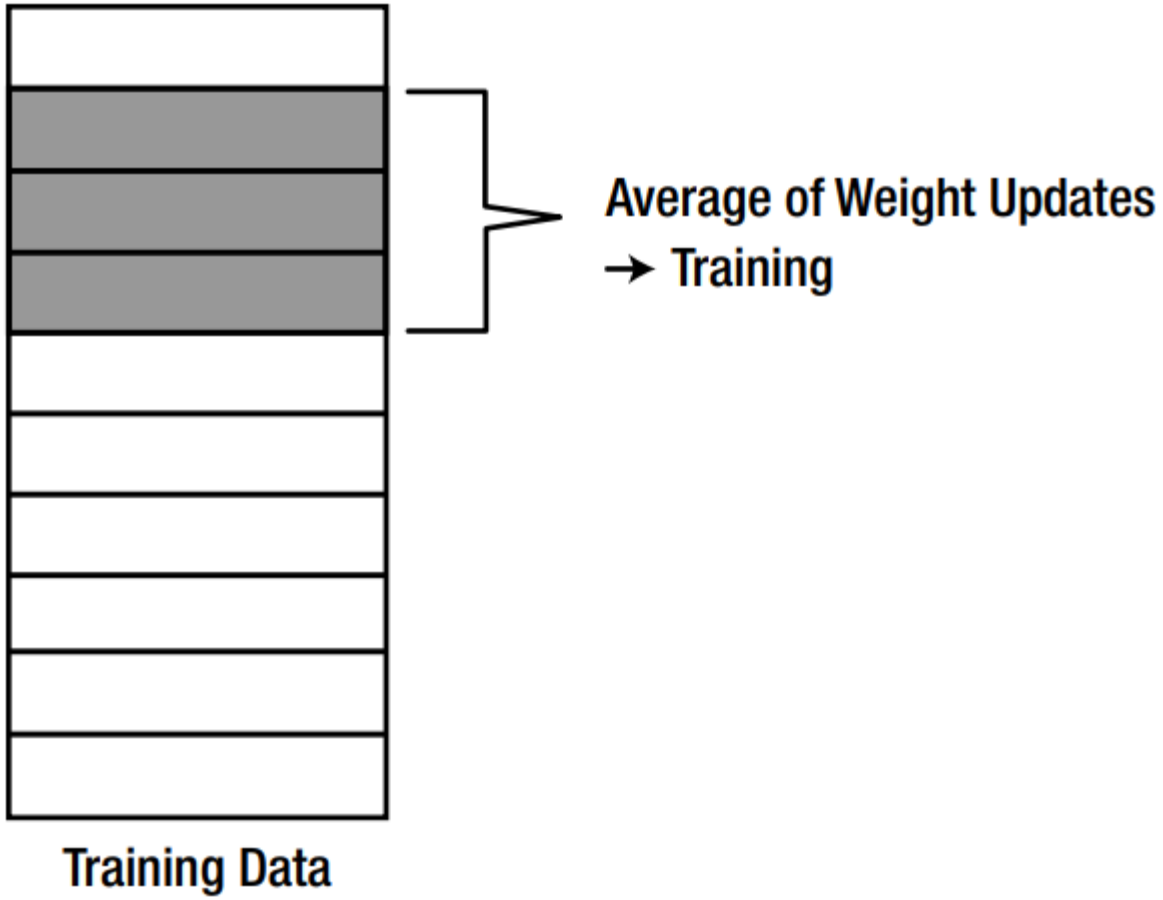
$$\Delta w_{ij} = \alpha \delta_i x_j$$

批次（Batch）方法：计算每一个训练数据的error，并根据均值更新权重。

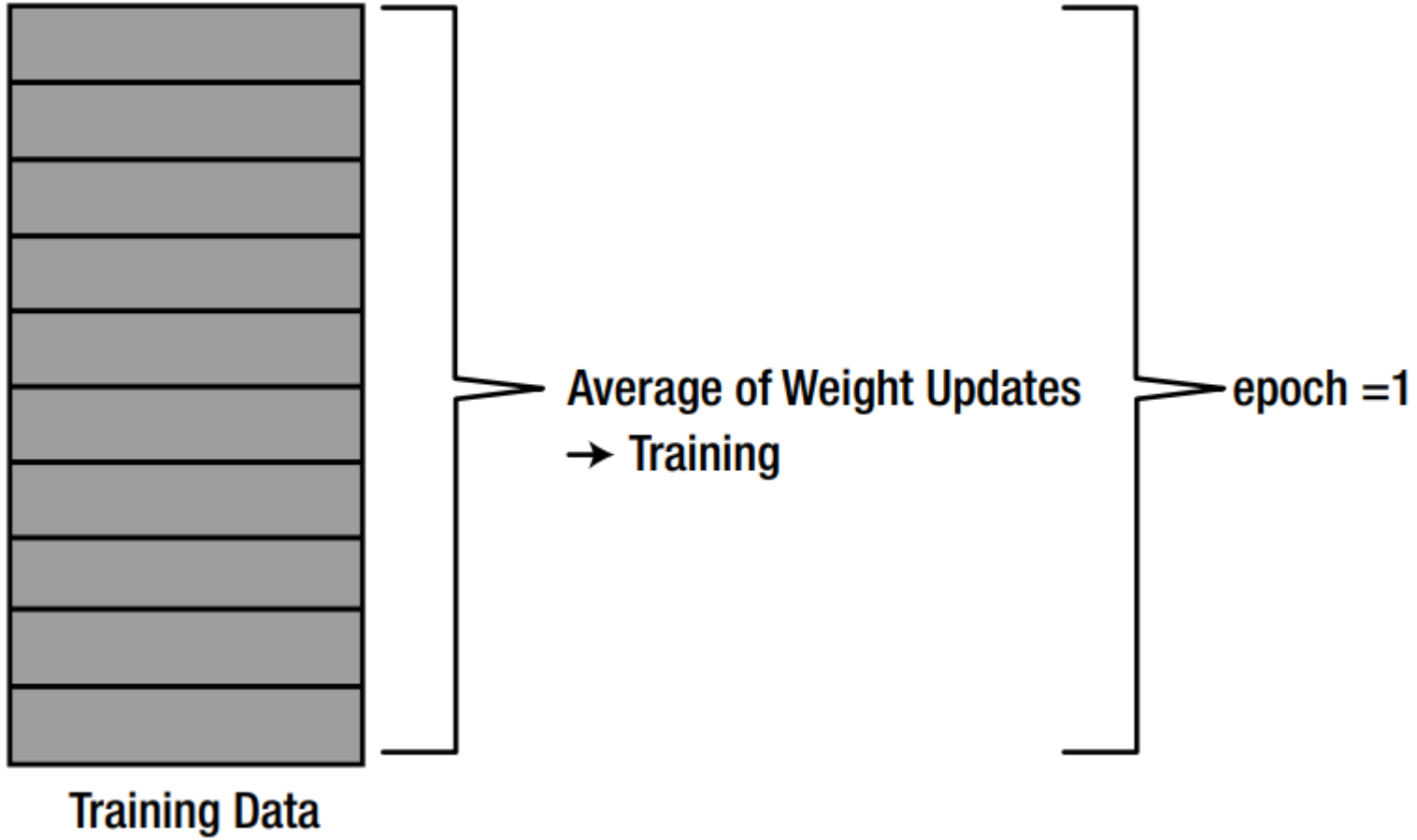


$$\Delta w_{ij} = \frac{1}{N}\sum_{k=1}^{N}\Delta w_{ij}(k)$$

Mini Batch：SGD和Batch的折中，每次调整weight是根据部分数据，知道所有数据都用完。

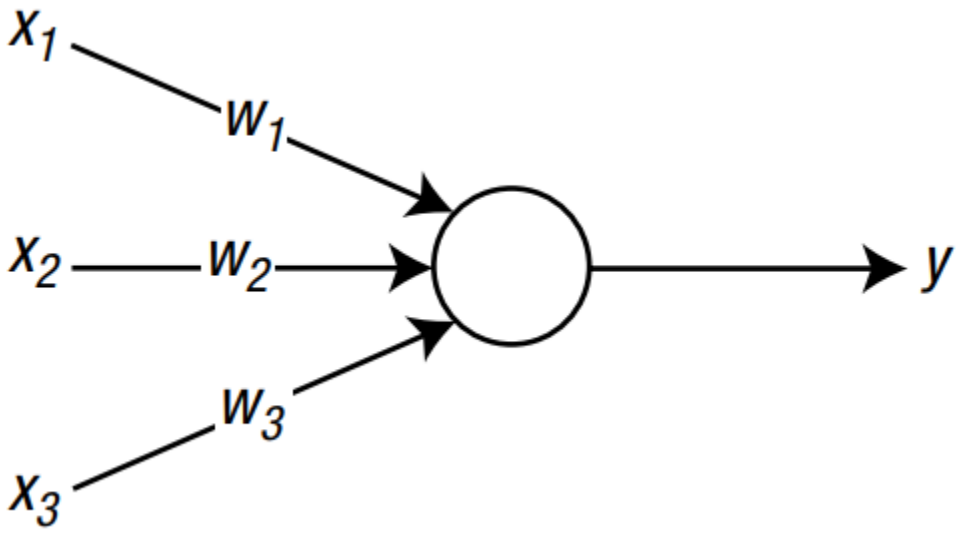


Epoch的概念：所有训练数据用一次

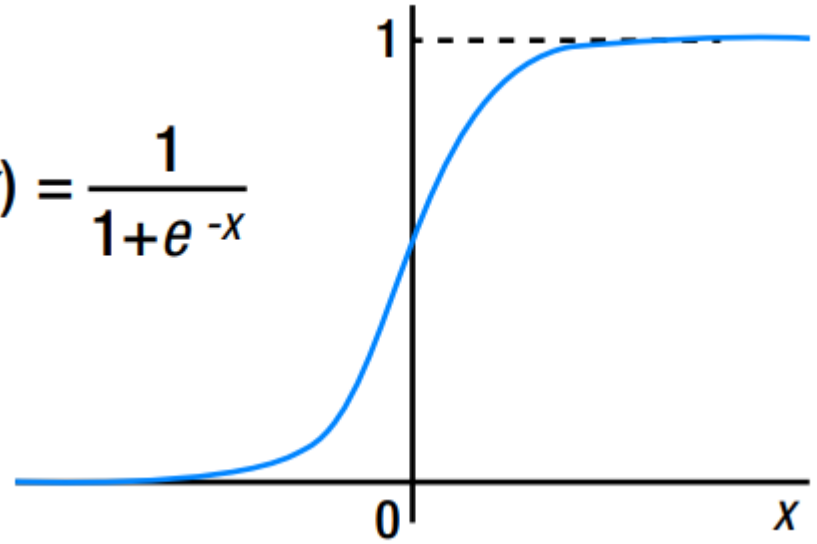


1.7 ： 实例Delta 规则训练

网络结构：考虑一个神经网络，3个节点的输入-1个节点输出， 激活函数为Sigmoid函数



$$\varphi(x) = \frac{1}{1+e^{-x}}$$



浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

训练数据：4组训练数据，其中前三位表示输入，最后标黑数字表示正确解（Ground-truth/Label）

$\{0, 0, 1, \mathbf{0}\}$
$\{0, 1, 1, \mathbf{0}\}$
$\{1, 0, 1, \mathbf{1}\}$
$\{1, 1, 1, \mathbf{1}\}$

更新规则：Delta 规则

$$\delta_i = \varphi(v_i)\left(1-\varphi(v_i)\right)e_i$$

$$\Delta w_{ij} = \alpha\,\delta_i\,x_j$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

实验5-1：

- 通过SGD训练方法及Delta 规则，对上述神经网络进行训练，并输出训练后的结果。
- 通过Batch训练方法及Delta 规则，对上述神经网络进行训练，并输出训练后的结果。
- 比较SGD训练方法及Batch训练方法误差(真实结果与输出的MSE)随epoch变化趋势，并可视化结果。
- 讨论不同alpha（学习率）对结果的影响。
- 讨论不同初始化W（权重）取值范围对结果的影响。

Step 1:定义Sigmoid函数

```
import numpy as np
def Sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))
```

Step 2:定义SGD函数

```
def DeltaSGD(W, X, D):  #D 表示准确解
    alpha = XXX
    N = 4  # 训练数据个数
    for k in range(N):
        x = X[k, :].T
        d = D[k]
        v = XXXXXX
        y = XXXXX
        e      = XXXXX
        W[0][0] = XXXXX
        W[0][1] = XXXXX
```

Step 3:初始化

```
import numpy as np
from Sigmoid import *
from DeltaSGD import *
X = XXXX
D = XXXXX
W = XXXXX
```

Step 4:训练并输出结果

```
for _epoch in range(10000):
    W = DeltaSGD(W, X, D)  #训练后的W

N = 4
for k in range(N):
      计算并打印出预测值y
```